Kernal Smoothers

Kernel Smoothers

Reading: You should read sections 6.1 to 6.5 in *The Elements of Statistical Learning*

There are a wide variety of methods for fitting a smooth curve through a set of x and y data when we don't know the functional form of the data. These methods are useful when we need a good approximation to the function underlying the points but don't care about understanding what the function is. One popular approach is **kernel smoothers**.

Consider the typical regression problem. For simplicity, we will limit ourselves to the case of a single explanatory variable.

We have n pairs of points (x_i, y_i) with

$$y_i = f(x_i) + \epsilon_i$$

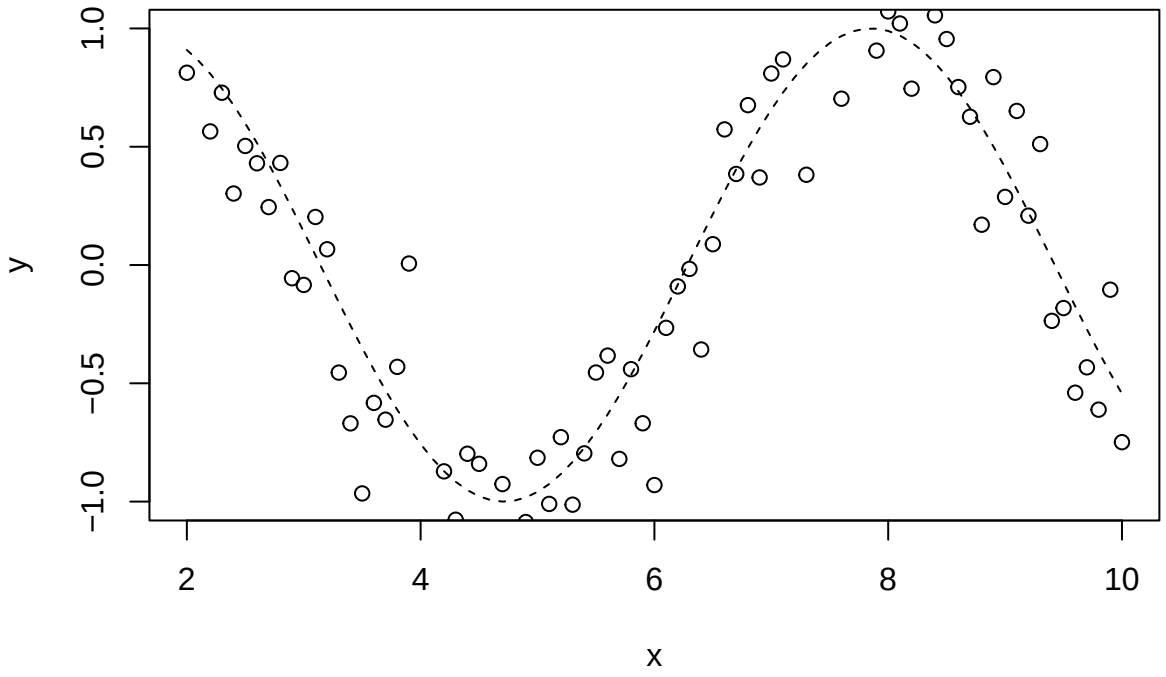
We want to estimate the function $f(x_i)$ at each point x_i. The basic idea of the kerne smoothers is that we estimate $f(x_i)$ by the mean (or some other function) of points in the neighborhood of x_i.

Consider the plot below. The points were generated as

$$y = sin(x) + \epsilon$$
$$\epsilon \sim normal(0, 0.03)$$

The line shows the sine curve. This is the true underlying non-linear function that we wouldn't know in a real data analysis situation. The points represent the data generated from the above model. Our task is to estimate the underlying curve given the data.

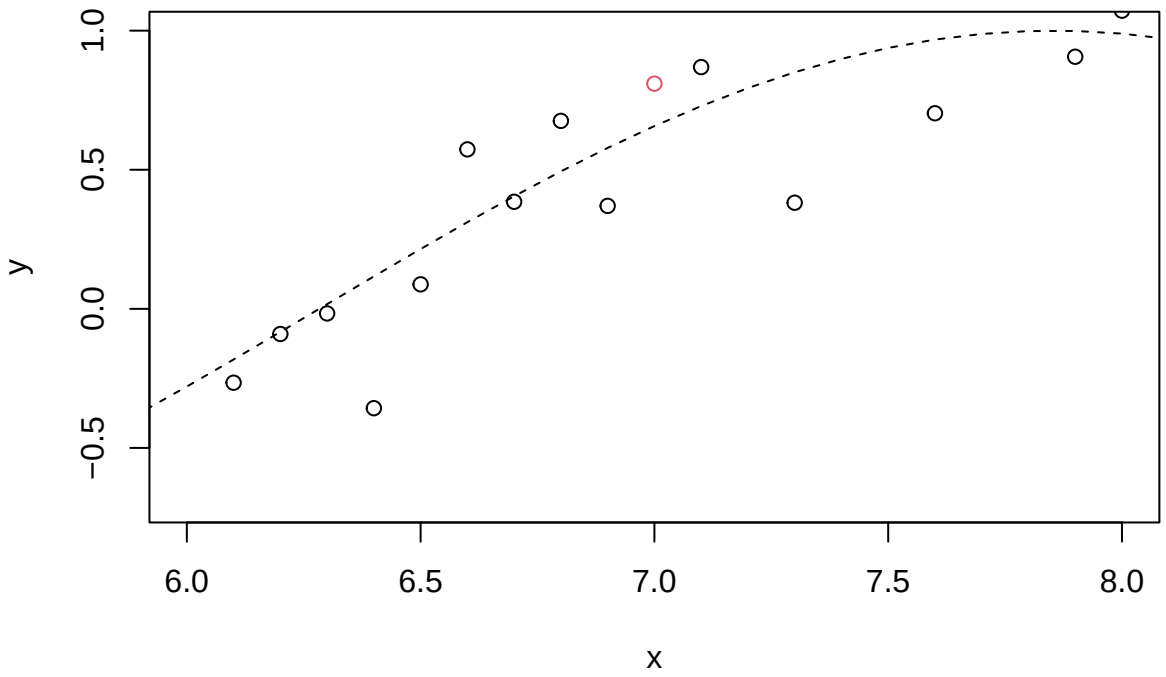
```
#code to produce non-linear regression data
set.seed(1211)   #this sets seed for the random number generator so
#that I will get the same "random" numbers every time

xpts<-seq(from=2,to=10,by=0.1)  #x point in the data
ypts<-sin(xpts)+rnorm(length(xpts),mean=0,sd=0.3)  #y points in the data are sin(x)+noise

plot(xpts,sin(xpts),type="l",lty=2,xlab="x",ylab="y") #plot the sine curve
points(xpts,ypts) #add in the data points
```




```
dat1=data.frame(xpts,ypts)
```

The plot below shows the same data, but zoomed in to the region x=6.0 to x=8.0:

```
plot(xpts,sin(xpts),type="l",lty=2,xlab="x",ylab="y",xlim=c(6,8),ylim=c(-0.7,1)) #plot the sine curve
points(xpts,ypts,col=ifelse(xpts==7.0,2,1)) #add in the data points
```



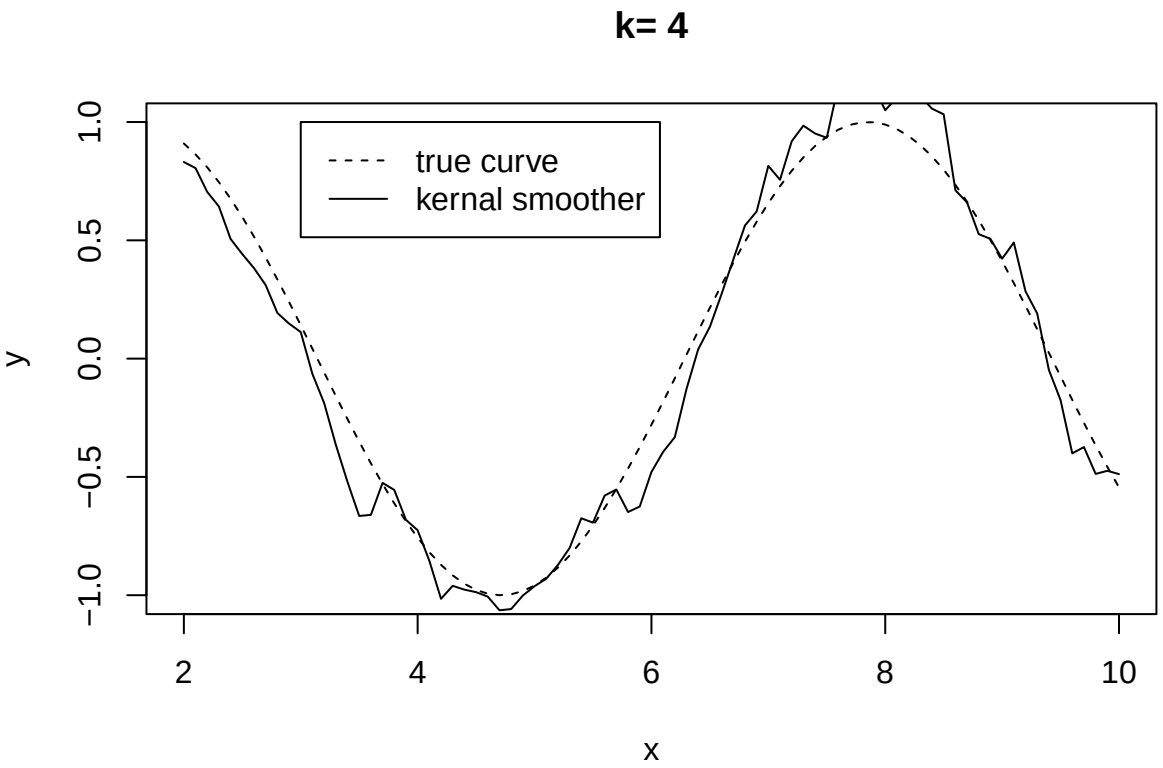
Consider the red point at x=7.0. This is somewhat above the true regression line. Thus, if we estimate the value of $f(7.0)$ as the y-value of this point, then we will get an estimate that is too high. However, the points nearby that point are roughly evenly split in being above or below the true curve. Thus, if we average them we should get something close to the true value.

In the simplest form of kernel smoother, we take

$$\hat{f}(x) = mean\,(y_i | x_i \in N_k(x))$$

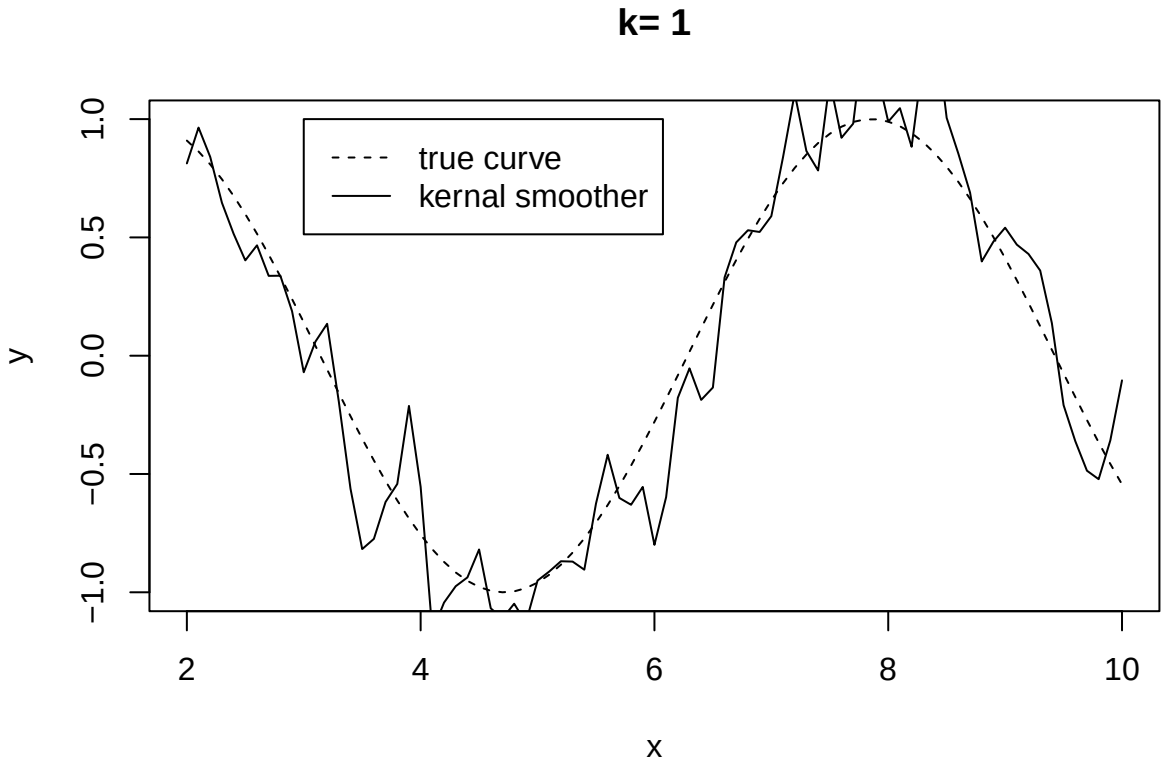
Where $N_k(x)$ is the set of k nearest points to x in squared distance. The program below does this.

```
#Program to do simple kernal smoothing by estimating the value of the curve with the mean of the
# k nearest neighbors:
par(mfrow=c(1,1))
fhat<-vector(length=nrow(dat1),mode="integer") #this will hold the estimates for the value of the curve

k<-4 #must be even

#loop over data points. For each data point, find the mean of y-values for point d and its k nearest ne
for(d in 1:length(xpts))
{   x1<-max(d-k/2,1)  #x1 and x2 are range over which to average points. the max prevents trying to use
    x2<-min(d+k/2,length(xpts))
    fhat[d]<-mean(ypts[x1:x2])  #take their mean
}

plot(xpts,sin(xpts),type="l",lty=2,xlab="x",ylab="y", main=paste("k=",k))
lines(xpts,fhat)
legend(3,1,c("true curve","kernal smoother"),lty=2:1)
```



The dashed line shows the true curve. The solid line shows the fitted values using the above method. This does quite well for such a simple method.

However, there is a problem with this – it will be discontinuous. Consider the case using $k = 1$:



Note that the fitted curve is jagged and discontinuous. The problem is that it assigns equal weight to the k-nearest points and zero weight to any other points. Thus, as points drop in and out of the k-nearest set then there are discrete jumps in the mean.

We avoid this using weights that decrease with distance from the point x. A popular method for kernel smoothing is the **Nadaraya-Watson kernel-weighted average**:

$$\hat{f}(x_0) = \frac{\sum_{i=1}^{n} K_\lambda(x_0, x_i) y_i}{\sum_{i=1}^{n} K_\lambda(x_0, x_i)}$$

$K_\lambda(x_0, x_i)$ is called the kernel function. This will be some smooth function of distance from x_0 that decreases as x_0 and x_i become further apart. At each value of x_0, we calculate a weighted mean of y-values. The weights decrease with distance so that nearby points contribute more to the mean than further points.

A standard approach is to use the following form for the kernel function:

$$K_\lambda(x_0, x_i) = D\left(\frac{|x - x_0|}{h_\lambda(x_0)}\right)$$

$$D(t) = \begin{cases} \frac{3}{4}(1 - t^2) & \text{if } |t| \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

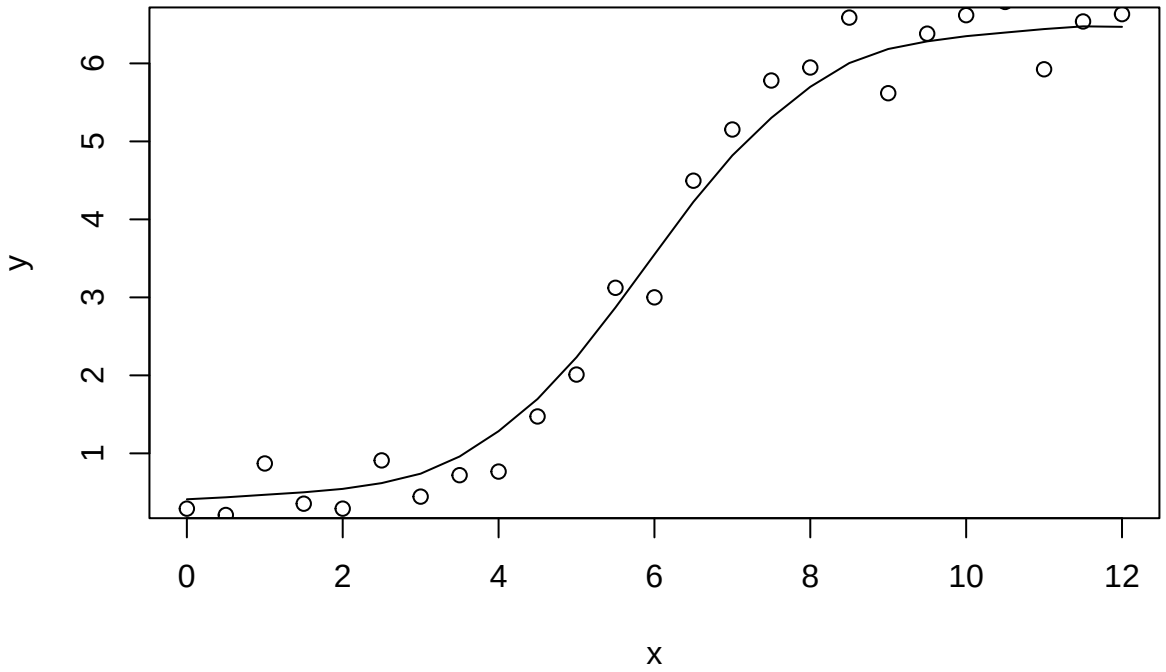
$h_\lambda(x_0)$ is a width function indexed by parameter λ . It determines how quickly the kernel function decreases with distance.

Here is a function that I wrote to do the Nadaraya-Watson kernel-weighted average:

```
NadWat<-function(x0,dat,lambda)
{   #a function to implement Nadaraya-Watson kernal-weighted average
    #estimates the weighted mean function at point x0

    x<-dat[,1]
    y<-dat[,2]
    n<-length(y)

    xdist<-mean(diff(x)) #mean distance between x points
    lambsc<-lambda*xdist #scale lambda by the mean distance between points

    klambda<-function(x0,x,lambda)
    {   t<-abs(x-x0)/lambda   #distance from x to x0, scaled by lambda
        return (ifelse(abs(t)<=1,0.75*(1-t*t),0)) #return zero if abs(t)>1
    }   #note that this gives a zero value for any point that is further than lambsc
        #from x0
        #Note that this function can be applied to a vector

    fhat<-(klambda(x0,x,lambsc)%*%y)/sum(klambda(x0,x,lambsc))   #returns sum(Ky)/sum(K)

    return(fhat)  #return weighted mean at x0
}
```

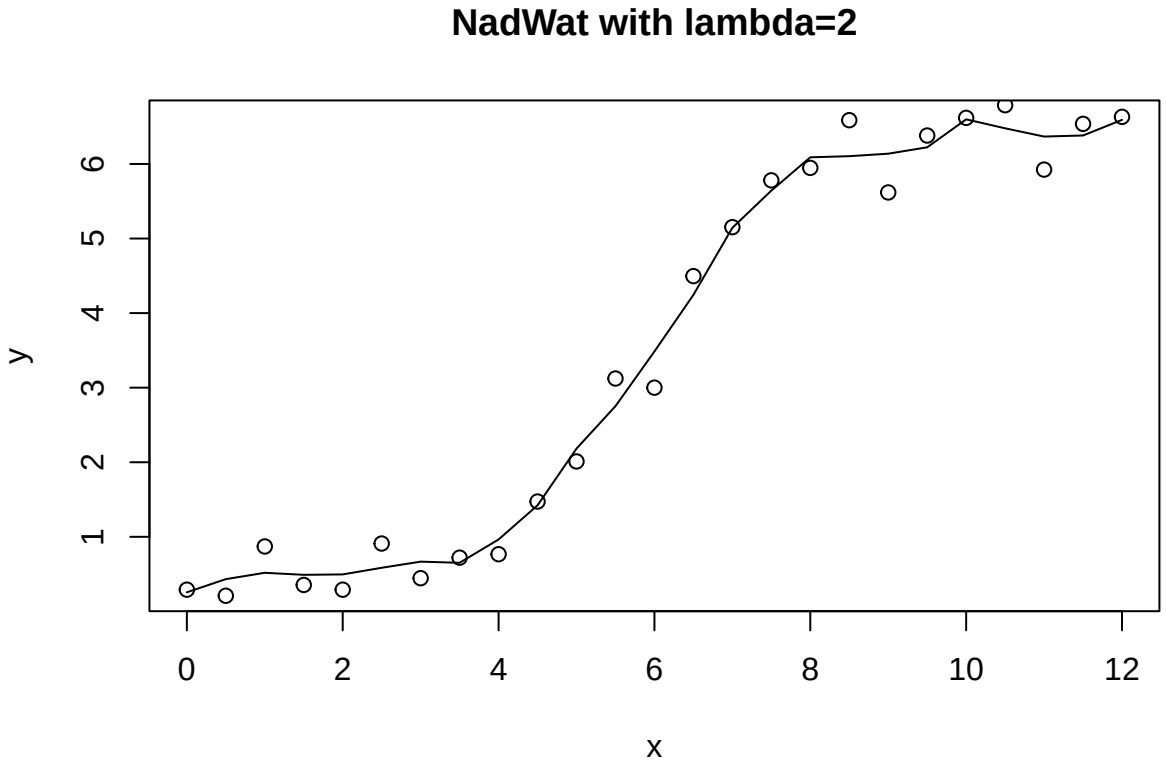
Apply this to the functional response data and make a plot:

```
dat1<-read.table("FuncResp.txt")

lambda=4.5  #will include points within distance=4.5*(mean distance between points)
x<-dat1[,1]
y<-dat1[,2]
n<-length(y)
fhatvals<-numeric(length=n)

for(i in 1:n)
    {   fhatvals[i]<-NadWat(x[i],dat1,lambda)
    }

#cbind(dat1[,2],fhatvals)
plot(dat1[,1],fhatvals,type="l",xlab="x",ylab = "y",main="NadWat with lambda=4.5")
points(dat1)
```



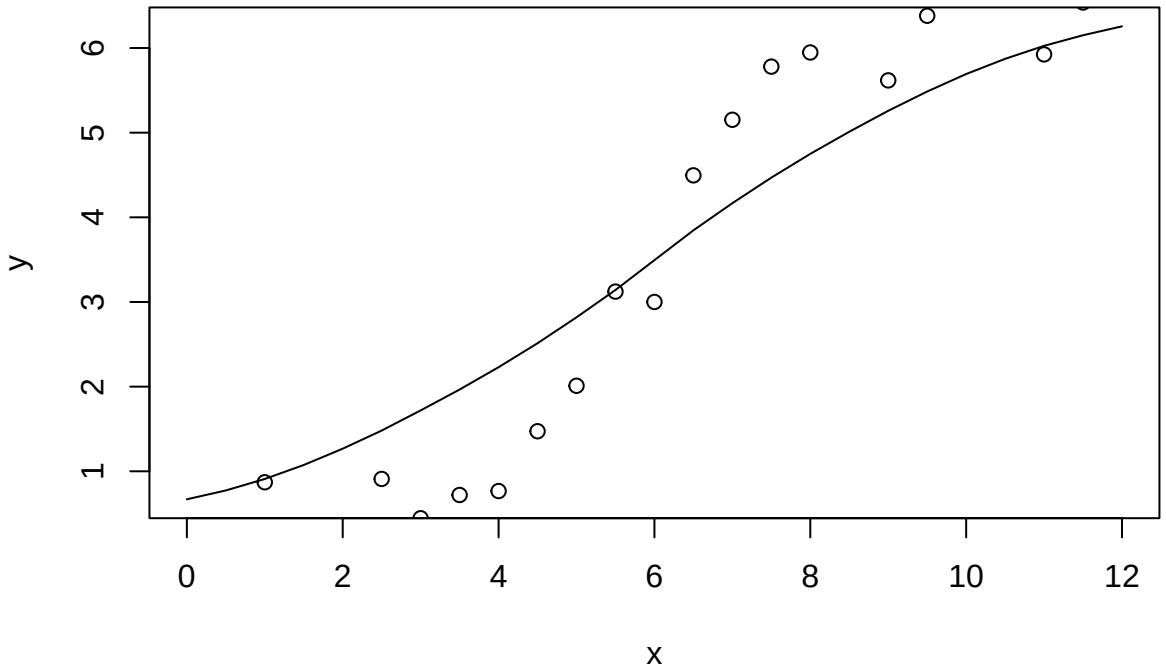
Too bumpy with lambda=2:

```
lambda=2
for(i in 1:n)
{   fhatvals[i]<-NadWat(x[i],dat1,lambda)
}

plot(dat1[,1],fhatvals,type="l",xlab="x",ylab = "y",main="NadWat with lambda=2")
points(dat1)
```



Too smooth with lambda=12:

```
lambda=12
for(i in 1:n)
{   fhatvals[i]<-NadWat(x[i],dat1,lambda)
}

plot(dat1[,1],fhatvals,type="l",xlab="x",ylab = "y",main="NadWat with lambda=12")
points(dat1)
```



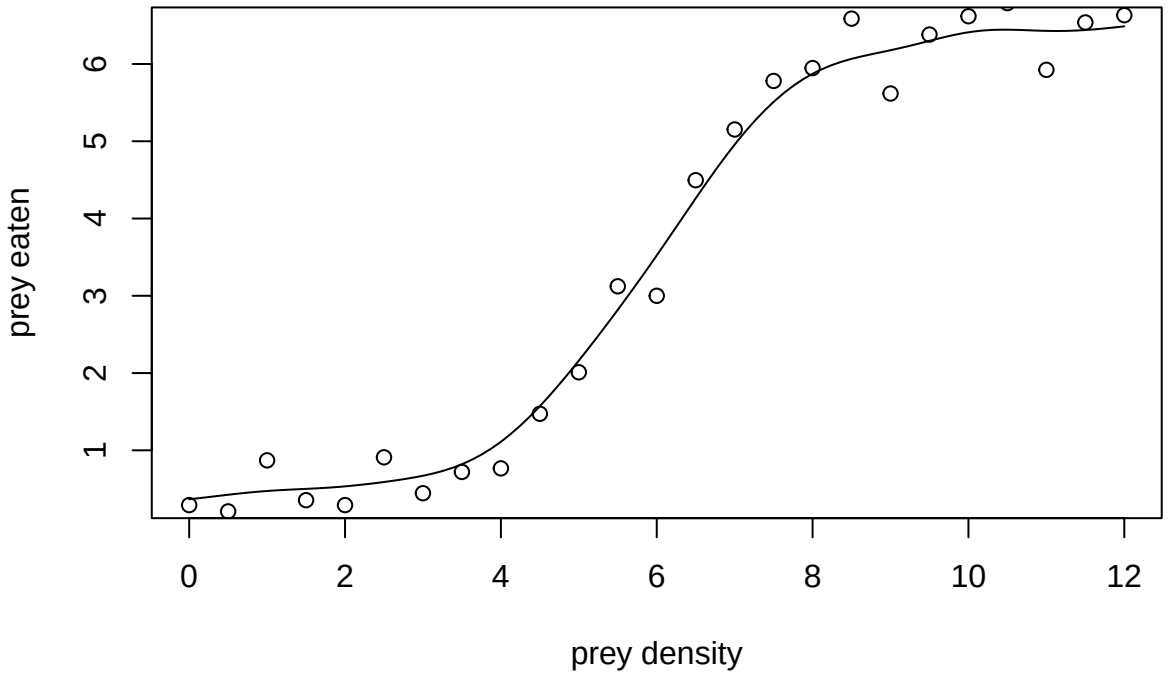
This method appears to do remarkably well given its simplicity. However, this method can run into problems at the ends because of the asymmetry of the fit there (i.e. points on only one side). This can also happen in the interior if the x points are not equally spaced.

The function `ksmooth` does this in R. Try it with the functional response data:

```
##Fit functional response using Nadaraya-Watson kernel smoother

dat1<-read.table("FuncResp.txt")
names(dat1)<-c("PreyDens","PreyEaten")
#plot(dat1[,1],dat1[,2], xlab="prey density", ylab="prey eaten")


#The function ksmooth() is a built in R function to do the same thing
?ksmooth
```

```
## starting httpd help server ... done
```

```
frks<-ksmooth(dat1[,1],dat1[,2],"normal",bandwidth=2)
plot(frks,type="l",xlab="prey density",ylab="prey eaten", main="functional response data with Nadaraya-W
points(dat1)
```



Exercise #1

Fit your data from the nonlinear regression exercises using `ksmooth` and plot. Use several different values of the bandwidth and compare.

Local Linear Regression

The above method fits a constant at each x point. We can improve on this by fitting a weighted least-squares line at each target x point x_0:

$$\min_{\alpha(x_0),\beta(x_0)} K_\lambda(x_0, x_i)\,[y_i - \alpha(x_0) - \beta(X_0)]^2$$

Then, the estimate at point x_0 is

$$\hat{f}(x_0) = \hat{\alpha}(x_0) + \hat{\beta}(x_0)x_0$$

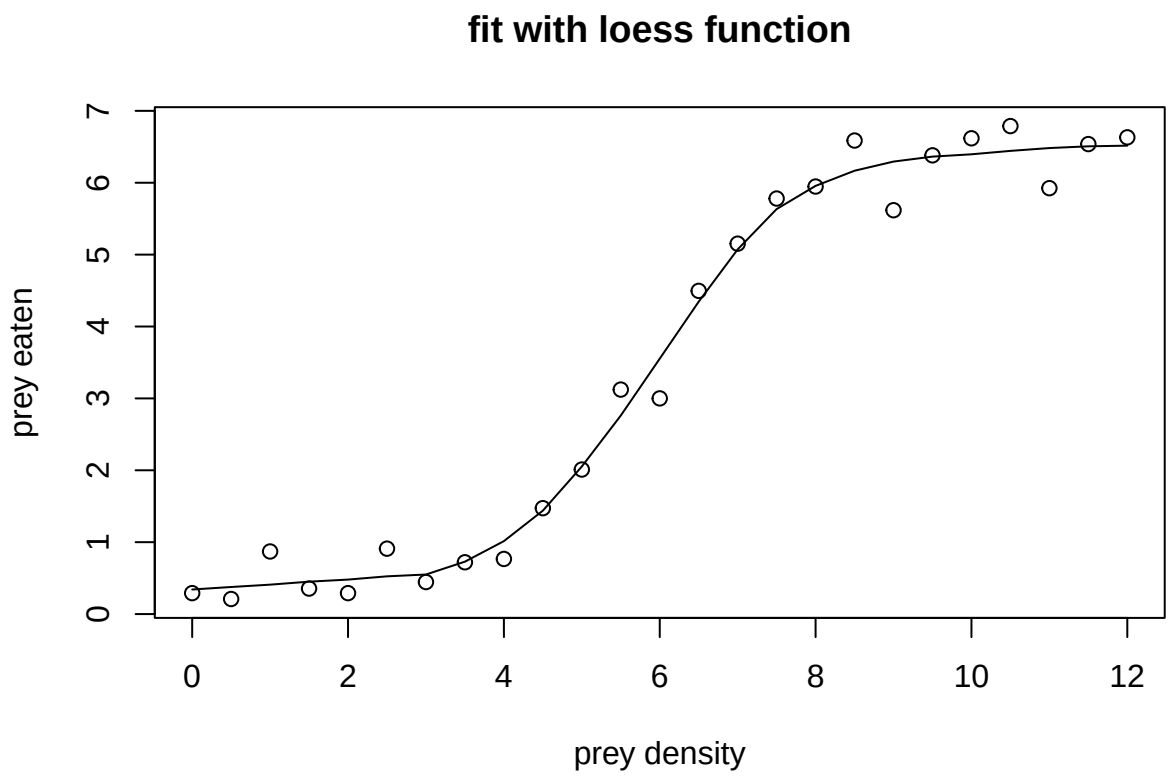
Note that although we use all of the data in the region to fit the line, we only evaluate it at the single point x_0. We do this at each x point.

The functions `loess` and `lowess` do local regression in R. `loess` is the newer version.

```
#apply this to the functional response data:
dat1<-read.table("FuncResp.txt")
names(dat1)<-c("PreyDens","PreyEaten")
plot(dat1[,1],dat1[,2],xlab="prey density",ylab = "prey eaten",main="fit with loess function")

lfr<-loess(PreyEaten~PreyDens,data=dat1, span=0.5)
summary(lfr)
```

```
## Call:
## loess(formula = PreyEaten ~ PreyDens, data = dat1, span = 0.5)
##
## Number of Observations: 25
## Equivalent Number of Parameters: 6.46
## Residual Standard Error: 0.3518
## Trace of smoother matrix: 7.14  (exact)
##
## Control settings:
##   span     :  0.5
##   degree   :  2
##   family   :  gaussian
##   surface  :  interpolate      cell = 0.2
##   normalize:  TRUE
##  parametric:  FALSE
## drop.square:  FALSE
```

```
lines(lfr$x,fitted(lfr))  #add fitted line to plot. lfr$x is x values that were fitted at. fitted(lfr)
```

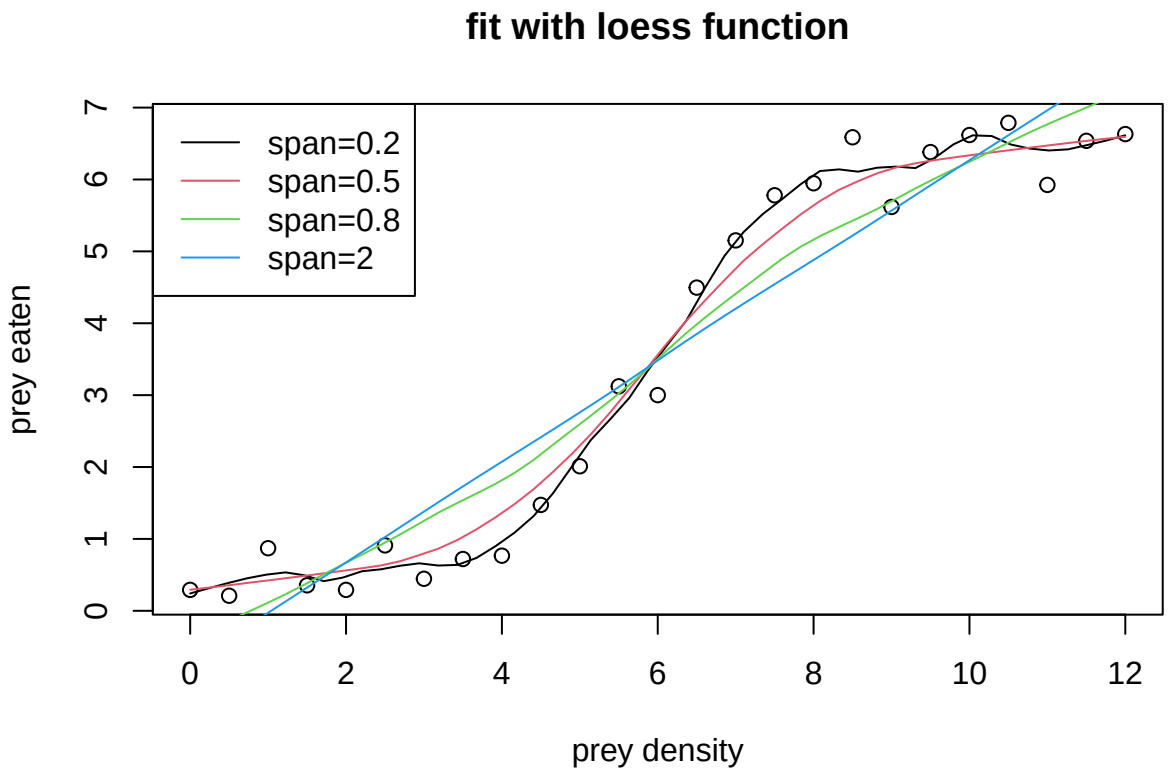



```
# fitted values from loess function
```

The plot shows the loess curve fitted to the functional response data. The parameter `span` determines the degree of smoothing – that is, how much further away points affect the estimate of the regression function at a given point.

```
#loess.smooth() is a function which evaluates the
#loess smooth function at equally spaced points covering the
#range of x
```

```
plot(dat1[,1],dat1[,2],xlab="prey density",ylab = "prey eaten",main="fit with loess function")
lines(loess.smooth(dat1[,1],dat1[,2],span=0.2),col=1)
lines(loess.smooth(dat1[,1],dat1[,2],span=0.5),col=2)
lines(loess.smooth(dat1[,1],dat1[,2],span=0.8),col=3)
lines(loess.smooth(dat1[,1],dat1[,2],span=2),col=4)
legend("topleft",c("span=0.2","span=0.5","span=0.8","span=2"),col=1:4,lty=1)
```



The plot above shows how the `span` parameter affects the fit. When `span` is small, then the fit is based only on nearby points and is thus more affected by random variation in the data. As the `span` parameter gets bigger, the fit at each point is more affected by further distant points. This decreases the effect of random variation and smooths out the curve. However, the curve can be over-smoothed. For example, the purple curve (`span`=2) is essentially a straight line. This is because it gives large weight to every point and eliminates too much variation between points.

Most methods of nonlinear regression that are not model based have a smoothing parameter that must be specified. Choosing the appropriate degree of smoothing is one of the most challenging aspects of nonlinear regression.

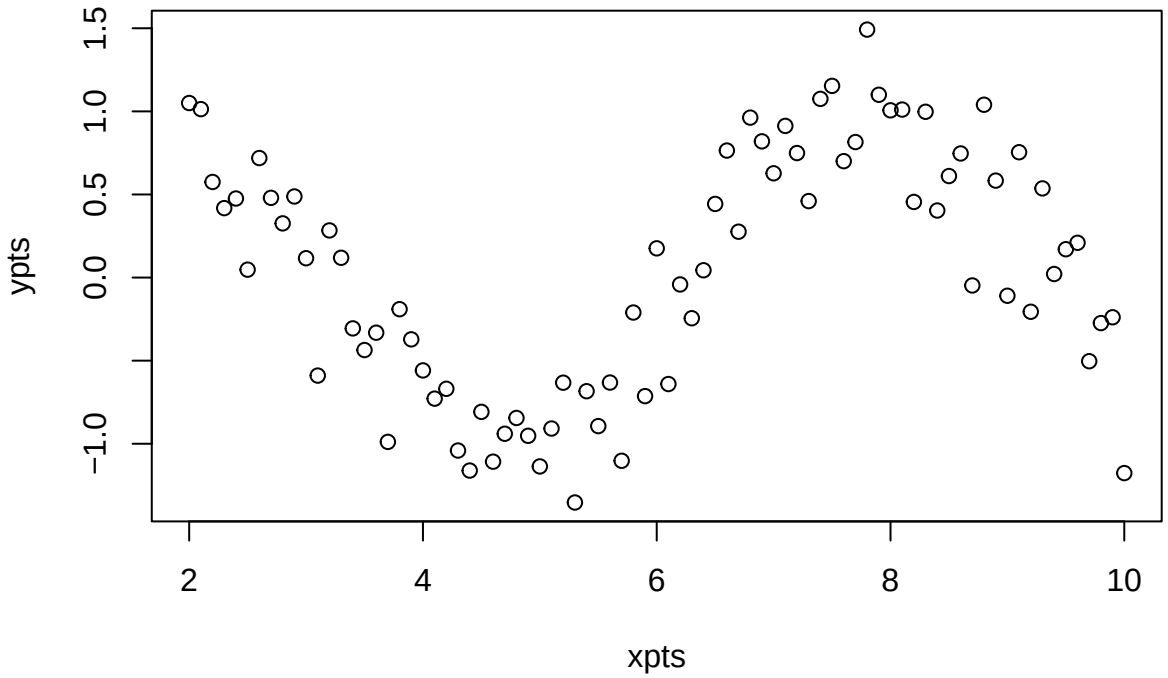
Cross Validation for Choosing the `span` parameter

We see above that the value of the span parameter makes a large difference in the fit. Next, we will go through an example of using cross validation to choose the span parameter.

The sample size of 25 for the functional response data is too small to use cross validation for choosing the span parameter. If we do 5-fold CV, this leaves only 5 data points per fold and there won't be enough points to estimate the value of the function in many intervals.

Instead, we use data simulated from the sine function as at the beginning of the notes.

```r
  #x points used to plot sin
set.seed(545)
xpts<-seq(from=2,to=10,by=0.1)  #x points that are data
ypts<-sin(xpts)+rnorm(length(xpts),mean=0,sd=0.3)  #y points in the data are sin(x)+noise

dat=data.frame(xpts,ypts)

plot(dat)
```



Here is code to use cross validation to find the optimal span paramter. Start with psuedocode:

define number of folds, span values to search, vector to store MSEs

assign folds to data points

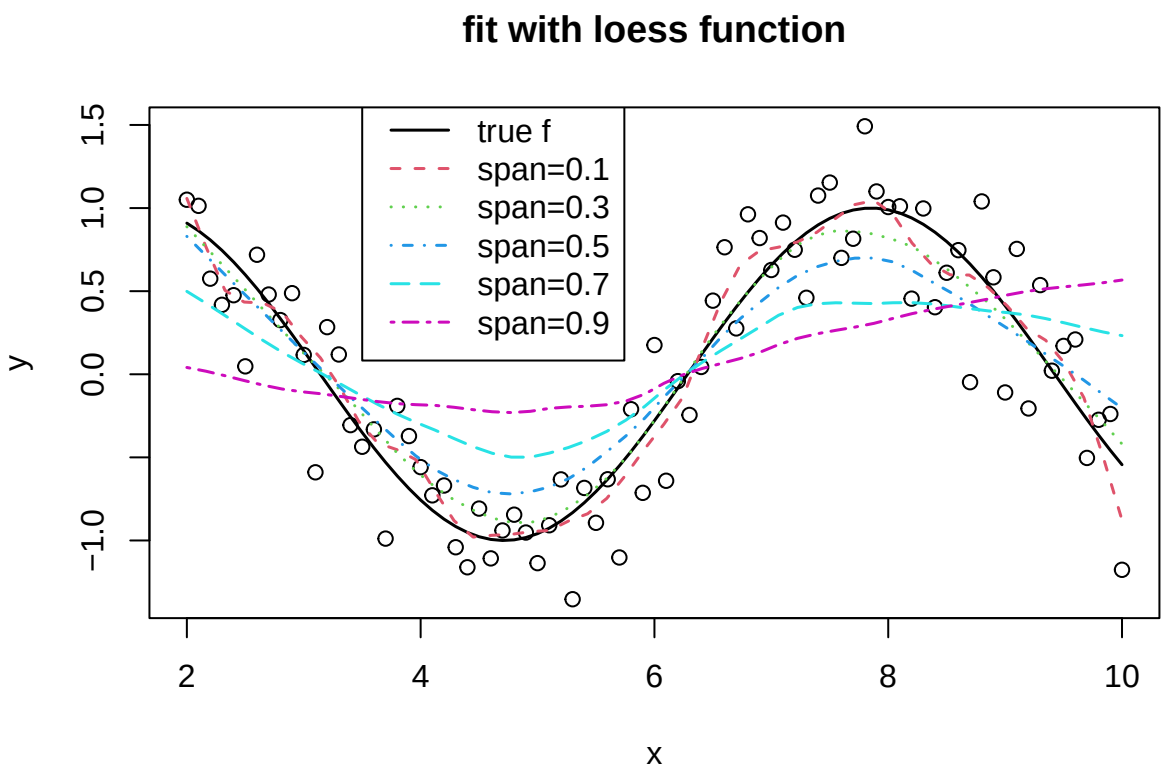
loop s over span values
{
loop f over folds
{ fit model to data with fold f left out
find predicted values in fold f from fitted model
find and store MSE between predicted and actual
}
find and store mean MSE over folds for current s
}

find s corresponding to lowest MSE

```r
##Code completed in class
```

From this plot, we see that a span of 0.4 produces the lowest MSE. Let's look at plots of the fitted curves versus the true cruve:

```
plot(dat[,1],dat[,2],xlab="x",ylab = "y",main="fit with loess function")
lines(xpts,sin(xpts), col=1, lty=1, lwd=1.5)
lines(loess.smooth(dat[,1],dat[,2],span=0.1),col=2, lty=2, lwd=1.5)
lines(loess.smooth(dat[,1],dat[,2],span=0.3),col=3, lty=3, lwd=1.5)
lines(loess.smooth(dat[,1],dat[,2],span=0.5),col=4,lty=4, lwd=1.5)
lines(loess.smooth(dat[,1],dat[,2],span=0.7),col=5,lty=5, lwd=1.5)
lines(loess.smooth(dat[,1],dat[,2],span=0.9),col=6,lty=6, lwd=1.5)
legend(3.5,1.7,c("true f","span=0.1","span=0.3","span=0.5","span=0.7","span=0.9"),col=1:6,lty=1:6, lwd=
```



The loess curve with a span of 0.5 gets the basic shape of the curve without responding to random variation.